

EE 601L: Neural Networks and Deep Learning Lab

LAB-10: Transformer Networks and Neural Machine Translation

1. Objective

1. To understand the architecture of the Transformer model in detail.
2. To implement key components of Transformers from scratch:
 - Self-Attention
 - Multi-Head Attention
 - Positional Encoding
 - Encoder and Decoder Blocks
3. To analyze how Transformers overcome limitations of RNNs and LSTMs.
4. To build a sequence-to-sequence Transformer model for language translation.
5. To translate a non-English article into English using the implemented Transformer.

2. Problem: Neural Machine Translation Using Transformer

Goal: Design and implement a Transformer-based sequence-to-sequence model to translate text from a non-English language to English.

Task 1: Dataset Preparation

Dataset Options:

- Multi30k Dataset
- IIT Bombay English-Hindi Dataset: you can find dataset here [IITB Parallel Corpus Resources](#)
- Any custom bilingual dataset

Steps:

1. Select a language pair (e.g., Hindi $\rightarrow$ English, Telugu $\rightarrow$ English).
2. Perform preprocessing:
 - Convert to lowercase
 - Remove noise
 - Tokenize sentences
3. Create vocabularies:
 - Source vocabulary (src_vocab)
 - Target vocabulary (tgt_vocab)
4. Add special tokens: <PAD>, <SOS>, <EOS>, <UNK>
5. Convert sentences to index sequences.
6. Apply padding to fixed sequence length.
7. Split dataset: 80% training, 20% testing.

Task 2: Transformer Architecture (From Scratch)

1. Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

2. Multi-Head Attention

- Split embeddings into multiple heads
- Apply attention in parallel
- Concatenate outputs

3. Positional Encoding

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

4. Encoder Block

- Multi-head self-attention
- Add & Layer Normalization
- Feedforward network
- Residual connections

5. Decoder Block

- Masked self-attention
- Encoder-decoder attention
- Feedforward network
- Residual connections

Implementation Requirements:

1. Do not use built-in Transformer modules.
2. Use only basic tensor operations and linear layers.
3. Implement:
 - Attention masks
 - Padding masks
 - Look-ahead masks

Task 3: Forward Pass and Training

1. Loss Function: Cross-Entropy Loss
2. Optimizer: Adam
3. Use teacher forcing during training
4. Shift target sequences appropriately
5. Train for at least 15–20 epochs
6. Plot training loss vs epochs

Task 4: Translation Task

Main Task:

1. Take an article in any non-English language
2. Feed it into the trained Transformer
3. Generate English translation

Evaluation Metrics:

1. BLEU Score
2. Qualitative comparison:
 - Original sentence
 - Predicted translation
 - Ground truth

3. Expected Observations

1. Transformers capture long-range dependencies
2. Parallel computation leads to faster training
3. Attention mechanism focuses on relevant tokens dynamically
4. Translation quality improves with more data and attention heads

4. Inference and Conclusion

1. Attention replaces recurrence in sequence modeling
2. Positional encoding enables order awareness
3. Advantages:
 - Parallelism
 - Better gradient flow
4. Limitations:
 - High computational cost
 - Requires large datasets

5. Submission Instructions

Students must submit:

1. Source code (modular implementation)
2. Training plots
3. Sample translations
4. Report containing:
 - Objective
 - Methodology
 - Observations
 - Conclusion

Important Note

Focus on correct implementation of Transformer internals. Translation quality is secondary. Clean and modular code will be evaluated. if you stuck at any point make the necessary assumptions and continue. As Transformers is not taught fully in the theory course do these tasks in parallel with course